

线性结构：链表 · 栈 · 队列

链表 (Linked List)

核心思想：不需要连续内存。每个节点存数据 + 下一节点的指针。

对比数组：数组 $O(1)$ 查找但 $O(N)$ 插入/删除；链表 $O(N)$ 查找但 $O(1)$ 插入/删除。

节点结构

```
typedef struct node {
    int data;
    struct node *next;
} Node;
```

头部插入：需要改 head 本身，传 Node**

```
void insertHead(Node** head, int x) {
    Node* p = createNode(x);
    p->next = *head;
    *head = p;          // 改main里的head
}
```

创建节点

```
Node* createNode(int x) {
    Node* p = (Node*)malloc(sizeof(Node));
    p->data = x;
    p->next = NULL;    // 必须初始化为NULL!
    return p;
}
```

易错点

插中间时先 `node->next=p` 后 `p->next=node->next` → p 指向自己形成环！

头部插入用 Node* 传参 → 改不了 main 的 head。

删除 (Delete)

删第一个值为 x 的节点。分两情况：删头 vs 删中间。一定要 free！

```
void deleteNode(Node** head, int x) {
    // Case 1: 删头节点
    if ((*head)->data == x) {    // 注意括号!
        Node* tmp = *head;
        *head = (*head)->next;
        free(tmp); return;
    }
    // Case 2: 删中间节点
    Node* p = *head;
    while (p->next && p->next->data != x)
        p = p->next;            // 必须判 p->next 非 NULL
    if (p->next == NULL) return; // 没找到
    Node* tmp = p->next;
    p->next = p->next->next;      // 绕过被删节点
    free(tmp);
}
```

建链：串 3 个节点

```
Node* head = createNode(10);
head->next = createNode(20);
head->next->next = createNode(30);
// 10 -> 20 -> 30 -> NULL
```

遍历 (Traversal)

```
void printList(Node* head) {
    Node* p = head;          // 用临时指针
    while (p != NULL) {
        printf("%d\n", p->data);
        p = p->next;
    }
}
```

易错点

Node* p; 只管声明不管分配 → 野指针。
遍历用 head 直接移动 → head 丢了再也找不到链表头。

插入 (Insert)

在 node 后面插入：注意顺序！先连新节点的 next，再改 node 的 next。

```
void insertAfter(Node* node, int x) {
    Node* p = createNode(x);    // 先建节点
    p->next = node->next;        // 新连后
    node->next = p;              // 前连新
    // 顺序反了链表会断！
}
```

易错点

*head->data 优先级错 → *(head->data)，应为 (*head)->data。

NULL 检查放在 p->next->data 后面 → 先崩了才判断。忘了 free。

x 不存在时 p->next->next 段错误。

双向循环链表

```
typedef struct node {
    struct node *llink; // 指向前一个
    int item;
    struct node *rlink; // 指向后一个
}
```

```
} Node;
```

性质: $\text{ptr} = \text{ptr} \rightarrow \text{llink} \rightarrow \text{rlink} = \text{ptr} \rightarrow \text{rlink} \rightarrow \text{llink}$

栈 (Stack) —LIFO

后进先出。操作: Push(入栈)、Pop(出栈)、Top(查看栈顶)。

数组实现

```
#define MAX 100
typedef struct { int data[MAX]; int top; } Stack;
void init(Stack* s) { s->top = -1; }
void push(Stack* s, int x) {
    if(s->top>=MAX-1)return;
    s->data[++(s->top)] = x; // 先+后放
}
int pop(Stack* s) {
    if(s->top== -1) return -1;
    return s->data[(s->top)--]; // 先取后-
}
int isEmpty(Stack* s) { return s->top== -1; }
```

易错点

push 忘判满, pop 忘判空。
data[s->top++] = x 后自增错 → data[-1]。

多项式 ADT

链表表示: $\langle e_i, a_i \rangle$ 按指数降序。
系数数组浪费空间 (稀疏多项式), 链表节省空间。

栈应用一: 后缀表达式求值

遇到数字 push, 运算符 pop 两个、算完 push 回去。
例: 6 2 / 3 - 4 2 * + = 8。

扫描	操作	栈
6	push 6	[6]
2	push 2	[6,2]
/	pop2,pop6→6/2=3,push	[3]
3	push 3	[3,3]
-	pop3,pop3→3-3=0,push	[0]
4	push 4	[0,4]
2	push 2	[0,4,2]
*	pop2,pop4→4*2=8,push	[0,8]
+	pop8,pop0→0+8=8,push	[8]

栈应用二: 中缀转后缀

运算符	栈内 ISP	栈外 ICP
(	0	6
+ -	3	2
* /	5	4
)	—	1

规则: $\text{ICP} > \text{ISP}$ 入栈, 否则弹栈直到条件满足。

```
void infixToPostfix() {
    Stack s; init(&s); char ch;
    while((ch=getchar())!='\n'){
        if(ch=='\t')continue;
        if(ch>='0'&&ch<='9') printf("%c",ch);
        else if(ch=='(') push(&s,ch);
        else if(ch==''){
            while(top(&s)!='(') printf("%c",pop(&s));
            pop(&s); // 弹掉(
        } else { // + - * /
            if(top(&s) != '(' && isp(top(&s)) >= icp(ch))
                printf("%c",pop(&s));
            push(&s,ch);
        }
    }
}
```

游标实现 (无指针)

全局数组 CursorSpace[], 索引 0 是自由链表头。
malloc: $\text{p} = \text{CursorSpace}[0].\text{Next}; \text{CursorSpace}[0].\text{Next} = \text{CursorSpace}[\text{p}];$
free: $\text{CursorSpace}[\text{p}].\text{Next} = \text{CursorSpace}[0].\text{Next};$
 $\text{CursorSpace}[0].\text{Next} = \text{p};$

```
    }
    while(!isEmpty(&s)) printf("%c",pop(&s));
}
```

易错点

忘了弹掉（。忘了最后弹空栈。
只写 push(ch) 没有 &s。

手算示例：(1+2)*3-4/(5+6)
→ 1 2 + 3 * 4 5 6 + / -

栈应用三：函数调用

每次函数调用创建**栈帧 (Stack Frame)**：返回地址、局部变量、旧帧指针。递归层数太深 → 栈溢出。
尾递归消除：尾递归可转为迭代。例：PrintList 尾递归 → while 循环。

队列 (Queue) —FIFO

先进先出。Enqueue(入队尾)、Dequeue(出队首)。

简单数组队列

```
typedef struct { int data[MAX]; int front,rear; } Queue;
void init(Queue* q) { q->front=0; q->rear=-1; }
void enq(Queue* q,int x) { q->data[++(q->rear)]=x; }
int deq(Queue* q) {
    if(q->front>q->rear)return -1;    // 判空
    return q->data[(q->front)++];
}
```

缺点： front 前面的空间永远浪费。

线性结构复杂度总结

操作	数组	单向链表	双向链表
查找第 k 个	$O(1)$	$O(N)$	$O(N)$
头部插入	$O(N)$	$O(1)$	$O(1)$
尾部插入	$O(1)$ (有 size)	$O(N)$ (无尾针)	$O(1)$
中间插入	$O(N)$	$O(1)$ (已知前驱)	$O(1)$
删除	$O(N)$	$O(1)$ (已知前驱)	$O(1)$

循环队列

数组当环用。front=rear 为空，(rear+1)%MAX==front 为满。
浪费一个格子区分空/满，最多存 MAX-1 个。

```
void init(Queue* q) { q->front=0; q->rear=0; }
void enq(Queue* q, int x) {
    if((q->rear+1)%MAX==q->front) return;    // 先判满
    q->rear=(q->rear+1)%MAX;
    q->data[q->rear]=x;
}
int deq(Queue* q) {
    if(q->front==q->rear)return -1;           // 判空
    int x=q->data[q->front];
    q->front=(q->front+1)%MAX;
    return x;                                // return放最后
}
```

易错点

init 写成 rear=-1（循环队列应为 0）。
先移 rear 再判满 → 万一满的，rear 动了但数据没放。
dequeue 里 return 后还有 front++ → 永远不会执行。
循环队列空用 front>rear → 应 front==rear。

链表实现栈/队列

栈：链表头插头删（头作栈顶）— $O(1)$ 。
队列：链表尾插头删（维护头尾指针）— $O(1)$ 。

层序遍历用队列

二叉树的层序遍历：根入队 → 出队访问 → 左右孩子入队 → 重复。